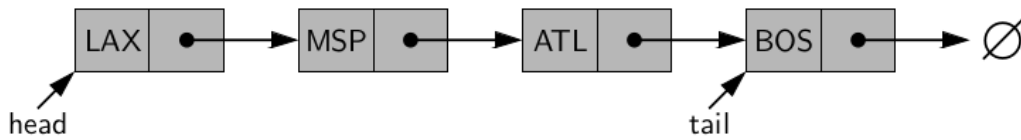


Linked Lists

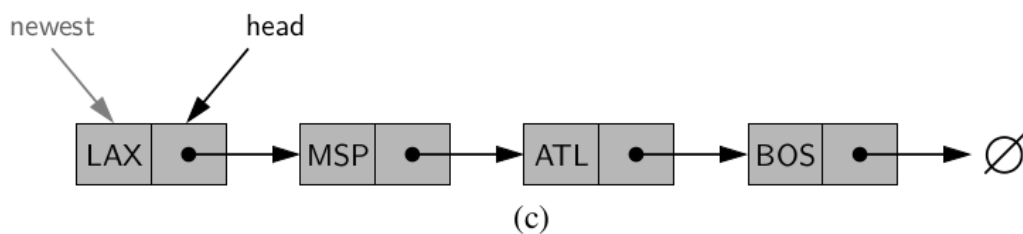
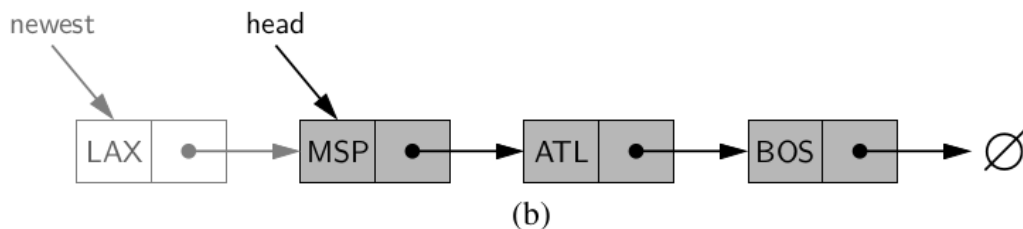
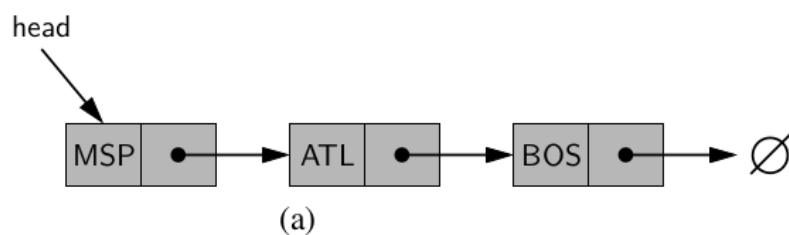
Limitations of array based implementations

- 1) The length is usually fixed apriori or if it is a dynamic array, it might be longer than the actual number of elements that it stores.
- 2) Amortized bounds for operations may be unacceptable in real-time systems
- 3) Insertions and deletions at interior positions of an array are expensive.

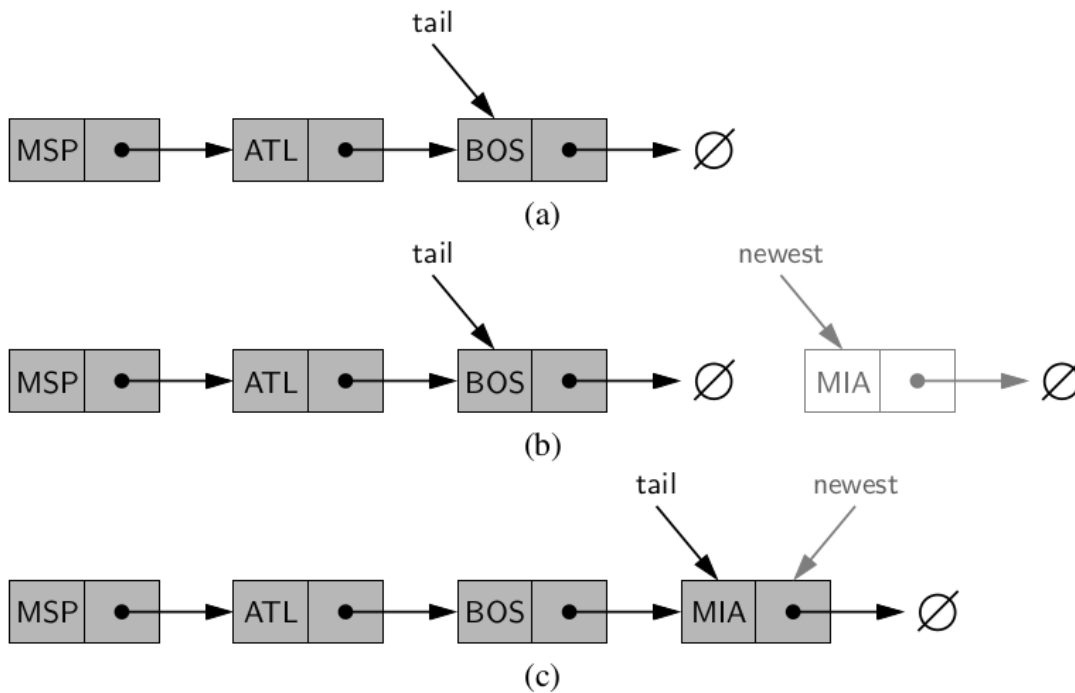
Singly Linked Lists



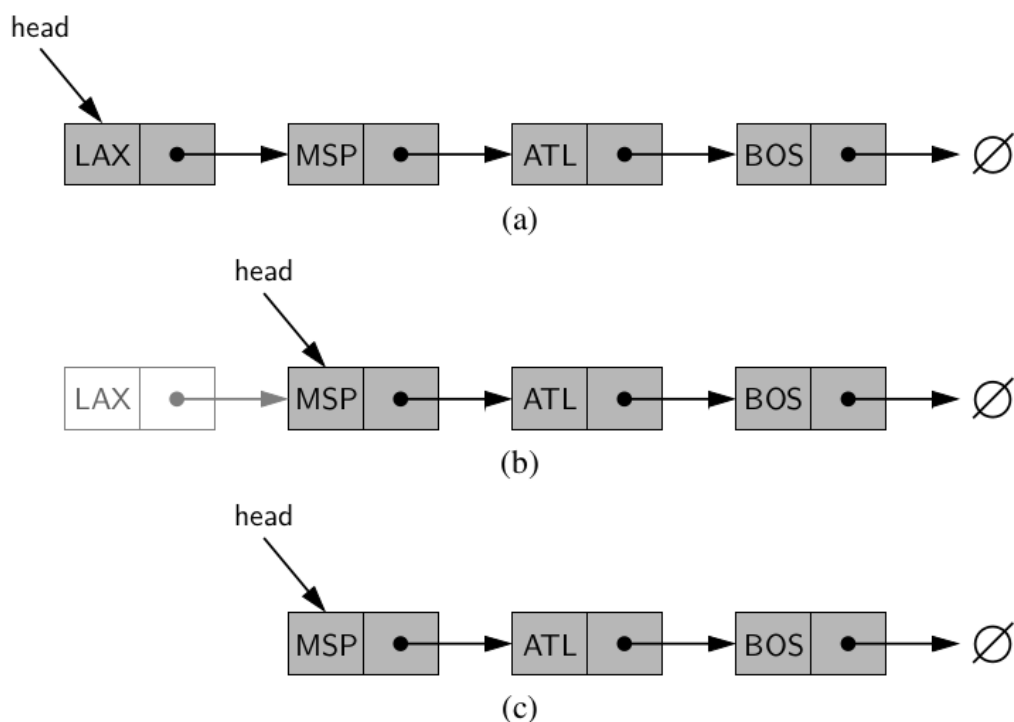
Inserting an Element at the Head of a Singly Linked List



Inserting an Element at the Tail of a Singly Linked List



Removing an Element from a Singly Linked List



The Node class

```
class _Node:
    """Lightweight, nonpublic class for storing a singly linked node."""
    __slots__ = '_element', '_next'      # streamline memory usage

    def __init__(self, element, next):    # initialize node's fields
        self._element = element          # reference to user's element
        self._next = next                 # reference to next node
```

```
1 class LinkedStack:
2     """LIFO Stack implementation using a singly linked list for storage."""
3
4     #----- nested _Node class -----
5     class _Node:
6         """Lightweight, nonpublic class for storing a singly linked node."""
7         __slots__ = '_element', '_next'      # streamline memory usage
8
9         def __init__(self, element, next):    # initialize node's fields
10             self._element = element          # reference to user's element
11             self._next = next                 # reference to next node
12
13     #----- stack methods -----
14     def __init__(self):
15         """Create an empty stack."""
16         self._head = None                    # reference to the head node
17         self._size = 0                       # number of stack elements
18
19     def __len__(self):
20         """Return the number of elements in the stack."""
21         return self._size
22
23     def is_empty(self):
24         """Return True if the stack is empty."""
25         return self._size == 0
```

Stack Implementation using a Singly Linked List

```
27 def push(self, e):
28     """Add element e to the top of the stack."""
29     self._head = self._Node(e, self._head)    # create and link a new node
30     self._size += 1
31
32 def top(self):
33     """Return (but do not remove) the element at the top of the stack.
34
35     Raise Empty exception if the stack is empty.
36     """
37     if self.is_empty():
38         raise Empty('Stack is empty')
39     return self._head._element                # top of stack is at head of list
```

Stack Implementation using a Singly Linked List

```
40 def pop(self):
41     """Remove and return the element from the top of the stack (i.e., LIFO).
42
43     Raise Empty exception if the stack is empty.
44     """
45     if self.is_empty():
46         raise Empty('Stack is empty')
47     answer = self._head._element
48     self._head = self._head._next            # bypass the former top node
49     self._size -= 1
50     return answer
```

Performance of the LinkedStack implementation

Operation	Running Time
<code>S.push(e)</code>	$O(1)$
<code>S.pop()</code>	$O(1)$
<code>S.top()</code>	$O(1)$
<code>len(S)</code>	$O(1)$
<code>S.is_empty()</code>	$O(1)$

All bounds are worst-case and the space usage is $O(n)$, where n is the current number of elements in the stack.

Queue Implementation using a Singly Linked List

```
1 class LinkedQueue:
2     """ FIFO queue implementation using a singly linked list for storage. """
3
4     class _Node:
5         """ Lightweight, nonpublic class for storing a singly linked node. """
6         (omitted here; identical to that of LinkedStack._Node)
7
8     def __init__(self):
9         """ Create an empty queue. """
10        self._head = None
11        self._tail = None
12        self._size = 0                # number of queue elements
13
14    def __len__(self):
15        """ Return the number of elements in the queue. """
16        return self._size
17
18    def is_empty(self):
19        """ Return True if the queue is empty. """
20        return self._size == 0
21
```

Queue Implementation using a Singly Linked List

```
21
22  def first(self):
23      """Return (but do not remove) the element at the front of the queue."""
24      if self.is_empty():
25          raise Empty('Queue is empty')
26      return self._head._element          # front aligned with head of list

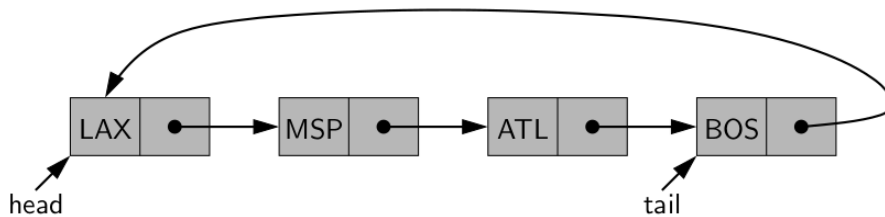
27  def dequeue(self):
28      """Remove and return the first element of the queue (i.e., FIFO).
29
30      Raise Empty exception if the queue is empty.
31      """
32      if self.is_empty():
33          raise Empty('Queue is empty')
34      answer = self._head._element
35      self._head = self._head._next
36      self._size -= 1
37      if self.is_empty():                 # special case as queue is empty
38          self._tail = None               # removed head had been the tail
39      return answer
```

Queue Implementation using a Singly Linked List

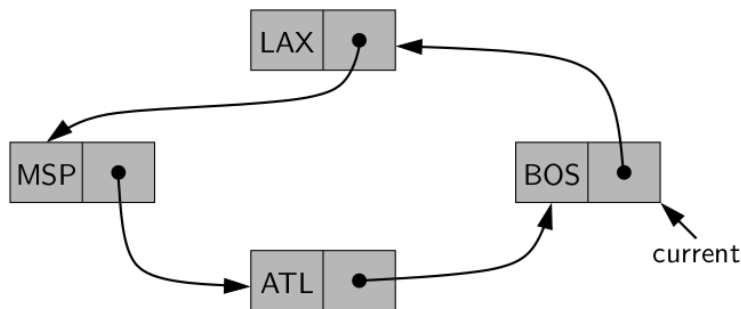
```
41  def enqueue(self, e):
42      """Add an element to the back of queue."""
43      newest = self._Node(e, None)        # node will be new tail node
44      if self.is_empty():
45          self._head = newest             # special case: previously empty
46      else:
47          self._tail._next = newest
48      self._tail = newest                 # update reference to tail node
49      self._size += 1
```

Circularly Linked Lists

Example of a singly linked list with circular structure



Example of a circular linked list, with current denoting a reference to a select node.



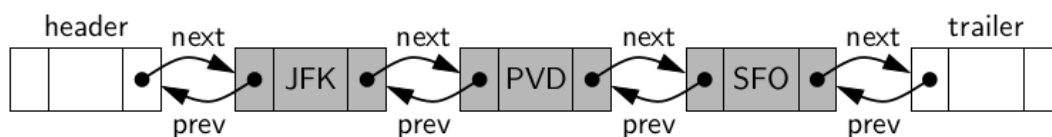
Circularly Linked Lists – key methods code walk-through

```
20 def first(self):
21     """Return (but do not remove) the element at the front of the queue.
22
23     Raise Empty exception if the queue is empty.
24     """
25     if self.is_empty():
26         raise Empty('Queue is empty')
27     head = self._tail._next
28     return head._element
29
30 def dequeue(self):
31     """Remove and return the first element of the queue (i.e., FIFO).
32
33     Raise Empty exception if the queue is empty.
34     """
35     if self.is_empty():
36         raise Empty('Queue is empty')
37     oldhead = self._tail._next
38     if self._size == 1:           # removing only element
39         self._tail = None       # queue becomes empty
40     else:
41         self._tail._next = oldhead._next  # bypass the old head
42     self._size -= 1
43     return oldhead._element
```

Circularly Linked Lists – key methods code walk-through

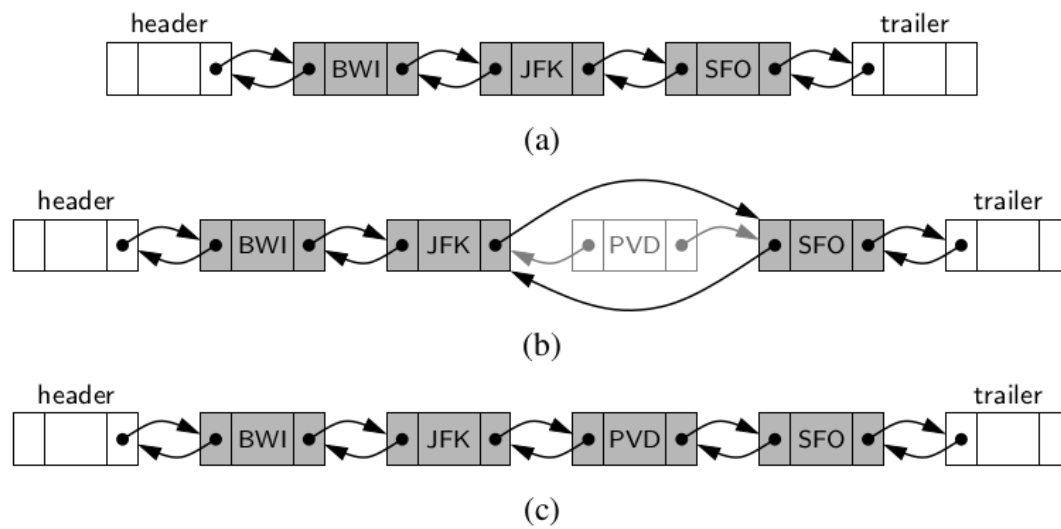
```
45 def enqueue(self, e):
46     """Add an element to the back of queue."""
47     newest = self._Node(e, None)      # node will be new tail node
48     if self.is_empty():
49         newest._next = newest          # initialize circularly
50     else:
51         newest._next = self._tail._next # new node points to head
52         self._tail._next = newest       # old tail points to new node
53         self._tail = newest             # new node becomes the tail
54     self._size += 1
55
56 def rotate(self):
57     """Rotate front element to the back of the queue."""
58     if self._size > 0:
59         self._tail = self._tail._next # old head becomes new tail
```

Doubly Linked Lists

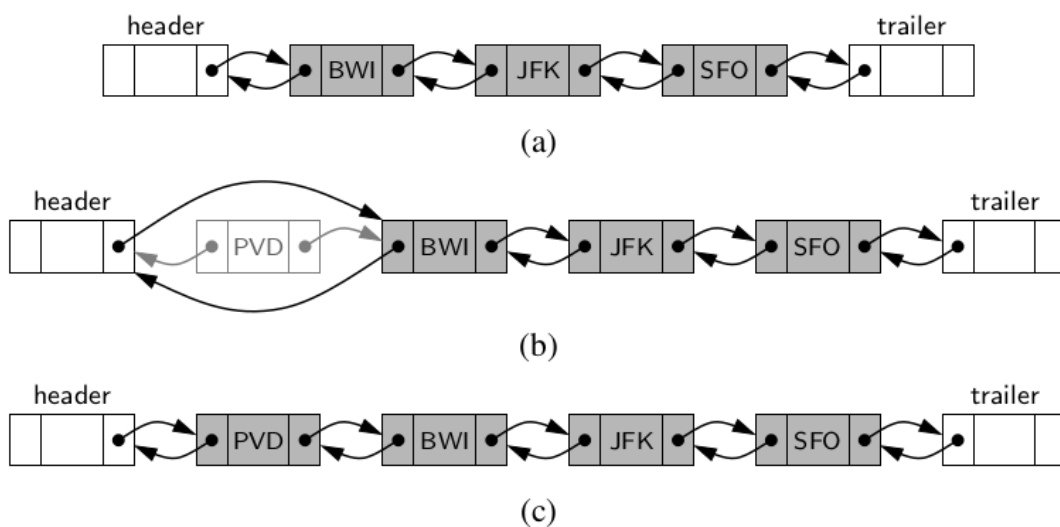


A doubly linked list representing the sequence { JFK, PVD, SFO }, using sentinels header and trailer to demarcate the ends of the list.

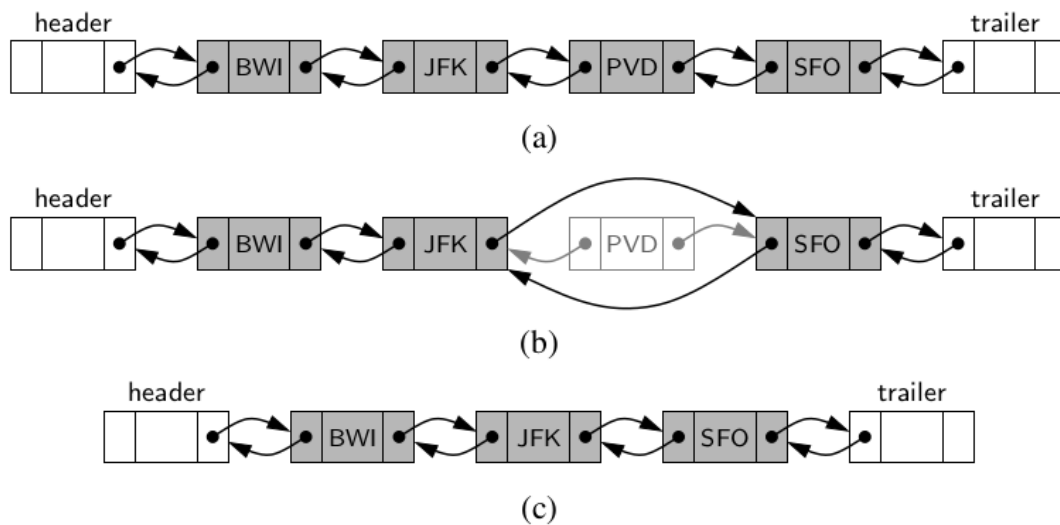
Adding an element to a doubly linked list (DLL) with header and trailer sentinels



Adding an element to the front of a DLL



Deleting an element from a DLL



The Positional List Abstract Data Type

A sequence of elements with the ability to identify the location of an element

Often implemented with a doubly linked list and supports operations like insertion, deletion, and access based on position in constant time $O(1)$

The Positional List Abstract Data Type – Basic Operations

L.first(): Return the position of the first element of L, or None if L is empty.

L.last(): Return the position of the last element of L, or None if L is empty.

L.before(p): Return the position of L immediately before position p, or None if p is the first position.

L.after(p): Return the position of L immediately after position p, or None if p is the last position.

L.is_empty(): Return True if list L does not contain any elements.

len(L): Return the number of elements in the list.

iter(L): Return a forward iterator for the *elements* of the list. See Section 1.8 for discussion of iterators in Python.

The Positional List Abstract Data Type – Basic Operations

The positional list ADT also includes the following *update* methods:

L.add_first(e): Insert a new element e at the front of L, returning the position of the new element.

L.add_last(e): Insert a new element e at the back of L, returning the position of the new element.

L.add_before(p, e): Insert a new element e just before position p in L, returning the position of the new element.

L.add_after(p, e): Insert a new element e just after position p in L, returning the position of the new element.

L.replace(p, e): Replace the element at position p with element e, returning the element formerly at position p.

L.delete(p): Remove and return the element at position p in L, invalidating the position.

The Positional List Abstract Data Type – Basic Operations

A position instance is a simple object, supporting only the following method:

p.element(): Return the element stored at position p.

Differences between Arrays and Linked Lists

Discussed in the class

Exercise problems

- Give an algorithm for finding the second-to-last node in a singly linked list in which the last node's next member points to a NULL.
- Describe a good algorithm for concatenating two singly linked lists L and M, given only references to the first node of each list, into a single list L that contains all the nodes of L followed by all the nodes of M.

Exercise problems

Suppose that *x* and *y* are references to nodes of circularly linked lists, although not necessarily the same list. Describe a fast algorithm for telling if *x* and *y* belong to the same list.

Our `CircularQueue` class of Section 7.2.2 provides a `rotate( )` method that has semantics equivalent to `Q.enqueue(Q.dequeue( ))`, for a nonempty queue. Implement such a method for the `LinkedQueue` class of Section 7.1.2 without the creation of any new nodes.

Exercise problems

Describe a nonrecursive method for finding, by link hopping, the middle node of a doubly linked list with header and trailer sentinels. In the case of an even number of nodes, report the node slightly left of center as the “middle.” (Note: This method must only use link hopping; it cannot use a counter.) What is the running time of this method?

Do the above problem with a singly linked list.

Exercise problems

Given the set of element $\{a, b, c, d, e, f\}$ stored in a list, show the final state of the list, assuming we use the move-to-front heuristic and access the elements according to the following sequence: (a, b, c, d, e, f, a, c, f, b, d, e).

References

Data Structures and Algorithms in Python

Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser

Introduction to Algorithms

Leiserson, Stein, Rivest, Cormen

Algorithms, 4th Edition

Robert Sedgewick and Kevin Wayne

Few Images from the internet